

Virtual Data Analyst

Návod na lokálne spustenie aplikácie

Akademický rok 2025/2026

Poznámka k dostupnosti

Nasadená verzia aplikácie nie je verejne dostupná, pretože beží iba v internom prostredí spoločnosti PwC. Z rovnakého dôvodu nie je možné odovzdať ani interný GenAI API kľúč alebo prístup k internému PwC LLM endpointu. Tento návod preto slúži najmä ako technický postup, ako by sa aplikácia dala lokálne spustiť pri dostupnosti vlastnej infraštruktúry.

Od používateľa mimo prostredia PwC sa neočakáva, že aplikáciu bude možné bez ďalšej konfigurácie plnohodnotne rozbehať. Na funkčné spustenie je potrebné zabezpečiť vlastný Azure LLM endpoint, vlastný API kľúč, lokálnu MongoDB inštanciu a testovaciu SQL databázu.

1 Požiadavky

Pred spustením aplikácie je potrebné mať nainštalované:

- **Git** na stiahnutie repozitára,
- **Docker Desktop** alebo Docker Engine s podporou Docker Compose,
- **MongoDB** spustenú samostatne, napríklad ako Docker kontajner,
- **GenAI API kľúč** a vlastný Azure LLM base URL endpoint používaný backendom.

Aplikácia pozostáva z troch hlavných služieb:

- frontend na adrese `http://localhost:5173`,
- backend API na adrese `http://localhost:8000`,
- Redis cache, ktorá sa pri Docker Compose spúšťa automaticky.

MongoDB sa v hlavnom `docker-compose.yml` nespúšťa ako samostatná služba. Používateľ si ju musí spustiť samostatne, napríklad ako Docker kontajner, alebo nastaviť pripojenie na vlastnú existujúcu MongoDB inštanciu.

2 Stiahnutie projektu

Projekt sa najprv naklonuje z GitHub repozitára:

```
git clone https://github.com/peter115342/Virtual_Data_Analyst-TP.git
cd Virtual_Data_Analyst-TP
```

V ďalších častiach sa predpokladá, že všetky príkazy sa spúšťajú z koreňového adresára projektu.

3 Konfigurácia backendu

Backend načítava citlivé konfiguračné hodnoty zo súboru `backend/.env`. V aktuálnom projekte tento súbor obsahuje iba identifikátory Azure Entra ID aplikácie a API kľúč pre LLM endpoint. Ostatné hodnoty, napríklad `FASTAPI_ENV`, `REDIS_URL` a `MONGODB_URL`, sa pri Docker Compose spustení nastavujú v súbore `docker-compose.yml` alebo majú predvolené hodnoty v backend konfigurácii. Kľúče a heslá sa nemajú commitovať do repositára.

Príklad vývojového súboru `backend/.env`:

```
# Azure Entra ID
# Pri lokálnom dev mode nie su tieto hodnoty podstatne,
# pretoze aplikacia pouziva Skip auth (dev mode).
AZURE_AD_CLIENT_ID=sem_vlozte_client_id
AZURE_AD_TENANT_ID=sem_vlozte_tenant_id

# LLM
API_KEY=sem_vlozte_vas_api_kluc
```

Premenné `AZURE_AD_CLIENT_ID` a `AZURE_AD_TENANT_ID` sú potrebné pri reálnom prihlásení cez Microsoft Entra ID. Pri lokálnom vývojovom režime nie sú rozhodujúce, pretože používateľ môže vo frontende zvoliť *Skip auth (dev mode)* a backend prijme vývojový token dev.

Premenná `API_KEY` je kľúč pre GenAI službu. Pôvodný projekt používa interný PwC Azure LLM endpoint, ktorý nie je dostupný mimo prostredia PwC. Base URL tohto endpointu nie je v našom `backend/.env`; backend ju berie z predvolenej hodnoty v konfigurácii `backend/app/config.py`. Mimo prostredia PwC je preto potrebné v súbore `backend/app/config.py` zmeniť riadok:

```
openai_base_url: str = "https://genai-sharedservice-emea.pwc.com/"
```

Hodnota sa má nahradiť vlastným Azure LLM endpointom. Konfiguračná premenná sa volá `OPENAI_BASE_URL`, pretože backend komunikuje cez OpenAI SDK a OpenAI-compatible API rozhranie; v tomto projekte však predstavuje base URL pre Azure LLM gateway, nie verejné OpenAI API. Pri lokálnom spustení mimo PwC je preto potrebné získať vlastný Azure LLM API kľúč a nastaviť vlastný Azure endpoint. V aktuálnej implementácii backend očakáva modely `azure.gpt-4.1`, `vertex_ai.gemini-2.5-flash` a `azure.text-embedding-3-small`; pri inom poskytovateľovi alebo inom nasadení môže byť potrebné upraviť názvy modelov v backend kóde.

4 Konfigurácia frontendu

Frontend číta premenné zo súboru `frontend/.env`. Pre lokálny vývoj stačí nastaviť adresu backendu a ponechať Entra hodnoty prázdne alebo vyplnené testovacími hodnotami. Keďže aplikácia beží cez Vite v režime `dev`, zobrazí sa možnosť *Skip auth (dev mode)*.

Príklad súboru `frontend/.env`:

```
VITE_API_BASE_URL=http://localhost:8000

# Pre lokalny dev mode nie je realna Entra konfiguracia nutna
VITE_AZURE_AD_CLIENT_ID=00000000-0000-0000-0000-000000000000
VITE_AZURE_AD_TENANT_ID=common
VITE_AZURE_AD_AUTHORITY=common
VITE_AZURE_AD_TOKEN_MODE=id_token

# Voliteľne, ak sa dev bypass používa mimo Vite dev režimu
VITE_ENABLE_DEV_AUTH_BYPASS=true
```

Ak sa používa reálne prihlásenie cez Microsoft Entra ID, hodnoty `VITE_AZURE_AD_*` musia zodpovedať registrovanej aplikácii v Entra ID a backend musí mať nastavené povolené tenanty a audiences.

5 MongoDB

MongoDB slúži na ukladanie histórie chatových relácií. Projekt pre MongoDB neobsahuje samostatný Docker Compose súbor, preto si ju používateľ musí spustiť sám. Najjednoduchšie lokálne spustenie je cez samostatný Docker kontajner:

```
docker run -d --name vda-mongo -p 27017:27017 mongo:7
```

Keďže backend sa spúšťa cez hlavný `docker-compose.yml` a MongoDB beží ako samostatný kontajner publikovaný na hostiteľskom počítači, backend kontajner sa na ňu pripája cez špeciálny hostiteľský názov:

```
MONGODB_URL=mongodb://host.docker.internal:27017
```

V súbore `docker-compose.yml` je táto hodnota nastavená priamo:

```
MONGODB_URL=mongodb://host.docker.internal:27017
```

Ak MongoDB beží inde, hodnotu `MONGODB_URL` v `docker-compose.yml` treba zmeniť podľa reálneho umiestnenia MongoDB. Napríklad:

```
# Vzdialena MongoDB
MONGODB_URL=mongodb://pouzivatel:heslo@server.example.com:27017
```

6 Spustenie aplikácie cez Docker Compose

Odporúčaný vývojový postup je spustiť frontend, backend a Redis cez hlavný Docker Compose súbor:

```
docker compose up --build
```

Tento príkaz spustí:

- frontend kontajner s Vite dev serverom na porte 5173,
- backend kontajner s FastAPI aplikáciou na porte 8000,
- Redis kontajner na porte 6379.

Po spustení je potrebné otvoriť:

```
http://localhost:5173
```

Na prihlasovacej obrazovke treba kliknúť na *Skip auth (dev mode)*. Frontend následne posielajú na backend hlavičku `Authorization: Bearer dev`. Backend ju akceptuje, pretože v `docker-compose.yml` je nastavené:

```
FASTAPI_ENV=development
```

7 Pripojenie testovacej databázy

Samotná aplikačná databáza sa nepripája cez `DATABASE_URL`, ale cez modálne okno vo frontende. Po vstupe do aplikácie treba kliknúť na tlačidlo pre pripojenie databázy a vyplniť typ databázy, host, port, používateľské meno, heslo a názov databázy alebo service name.

Projekt podporuje hodnoty typov databázy:

- postgres,
- mysql,
- oracle,
- sqlserver.

V repozitári sú pripravené samostatné Docker Compose súbory pre testovacie databázy. Pre okamžité vyskúšanie aplikácie je pripravený jednoduchý dataset iba v PostgreSQL konfigurácii `docker-compose.db.yml`. Táto databáza sa spustí príkazom:

```
docker compose -f docker-compose.db.yml up -d
```

Keď aplikácia beží cez Docker Compose a testovacia databáza beží na hostiteľskom počítači alebo cez samostatný compose súbor s publikovaným portom, do poľa *Database Host* sa spravidla zadáva `host.docker.internal`.

Ostatné databázové typy sú podporované implementáciou backendu a vo frontende ich možno vybrať v modálnom okne pripojenia. Ak chce používateľ vyskúšať ďalšiu PostgreSQL databázu, MySQL, Oracle Database alebo SQL Server na vlastných dátach, musí do príslušného priečinka v adresári `db/` doplniť vlastné inicializačné skripty alebo dáta. Pre tieto databázy slúžia existujúce compose súbory najmä ako základná šablóna prostredia:

```
docker compose -f docker-compose.db_bikes.yml up -d
docker compose -f docker-compose.db_ecommerce.yml up -d
docker compose -f docker-compose.db_sqlserver.yml up -d
docker compose -f docker-compose.db_oracle.yml up -d
```

Databáza	Údaje pre lokálny test
PostgreSQL	typ postgres, port 5432, user postgres, heslo postgres, DB virtual_data_analyst

Tabuľka 1: Pripravená lokálna testovacia databáza

Naraz môže byť problém spustiť viac databáz používajúcich rovnaký port. V takom prípade treba spustiť iba jednu z nich alebo upraviť mapovanie portov.

8 Overenie funkčnosti

Po spustení aplikácie sa odporúča skontrolovať:

1. frontend na adrese `http://localhost:5173`,
2. prihlásenie cez *Skip auth (dev mode)*,
3. pripojenie testovacej databázy cez modálne okno,
4. odoslanie jednoduchkej otázky, napríklad *Kolko záznamov je v tabuľke?*.

Ak backend hlási, že MongoDB nie je pripojená, treba skontrolovať hodnotu `MONGODB_URL`. Ak zlyhá generovanie odpovede cez LLM, najčastejšou príčinou je chýbajúci alebo neplatný `API_KEY`. Ak sa aplikácia nevie pripojiť k testovacej databáze, treba overiť host, port, prihlasovacie údaje a to, či sa databáza pripája z kontajnera alebo priamo z hostiteľského počítača.

9 Ukončenie služieb

Hlavnú aplikáciu spustenú cez Docker Compose možno zastaviť príkazom:

```
docker compose down
```

Samostatnú testovaciu databázu možno zastaviť rovnakým spôsobom s príslušným compose súborom, napríklad:

```
docker compose -f docker-compose.db.yml down
```

Ak sa MongoDB alebo Redis spustili cez `docker run`, zastavia sa príkazmi:

```
docker stop vda-mongo  
docker stop vda-redis
```